TECHNION | The Henry and Marilyn Taub Faculty of Computer Science | CS4PEOPLE

Detecting Leg Motions for Robotic Hand Control

Winter 2026

—

Project in Machine Learning - 2360757

—

Instructor: Ori Bryt

Supervisor: Dean Zadok

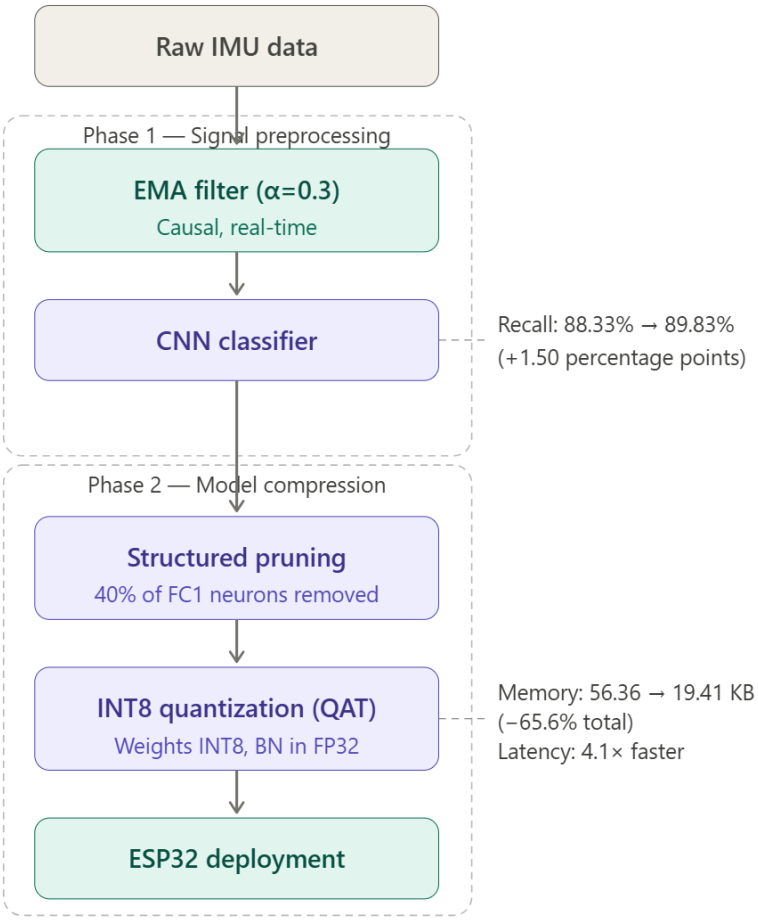
Students:

Abed-Al-Rahman Badran 325424752

Zina Assi 213813165

Abstract

This report presents our work on improving the Smart Ankleband system which is an assistive technology developed by Zadok et al. that enables people with upper-limb impairments to control a prosthetic robotic hand through ankle gestures detected by an IMU sensor. We investigated two complementary enhancements to the system: signal preprocessing through IMU data filtering and model compression through structured pruning and quantization. The optimal filtering solution we found is an exponential moving average (EMA) filter with α=0.3, it improved gesture recall by 1.50 percentage points while remaining suitable for real-time deployment. The model compression pipeline achieved a 65.6% reduction in memory footprint and a 4.1x inference speedup, while maintaining classification performance within acceptable bounds.



1. Introduction

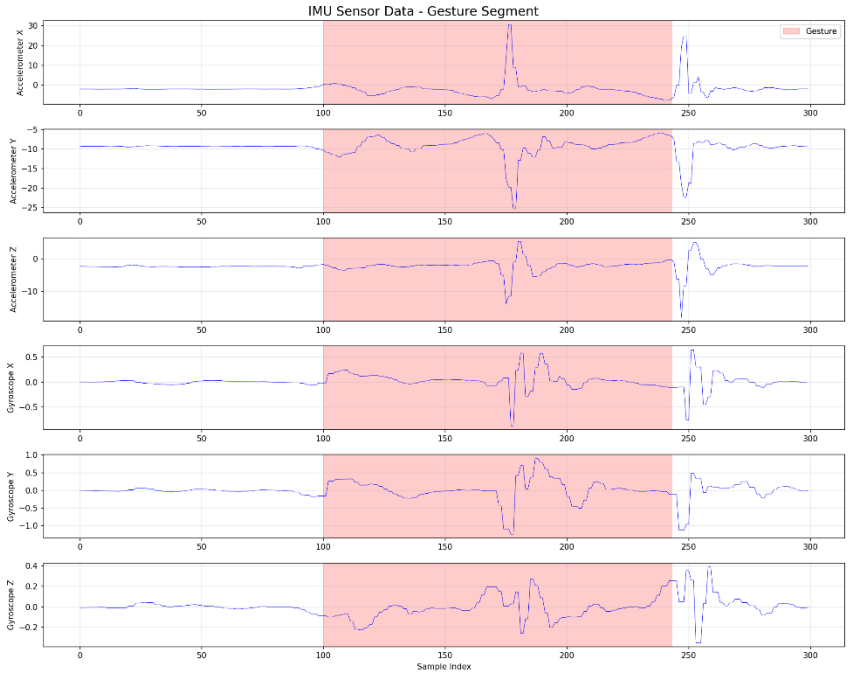
1.1 Background

The Smart Ankleband is an assistive technology developed by Zadok et al. [1] that allows people with upper-limb differences to control a prosthetic robotic hand using ankle movements. An IMU sensor mounted on the ankle captures the user's leg gestures, which are classified in real time by a deep learning model and translated into hand commands. This approach offers a non-invasive, affordable alternative to muscle-based prosthetic control systems.

The sensor used is an Adafruit BNO08X IMU, recording 6-axis motion data with three axes of acceleration and three axes of gyroscope readings at 200 Hz. A lightweight 1D CNN with approximately 14,000 parameters runs on an ESP32 microcontroller, classifying five gesture types plus a rest state under a strict 90KB memory constraint.

1.2 Dataset and Baseline

The dataset was collected from 10 subjects performing ankle gestures in both seated and standing positions, totaling approximately 2.5 million timesteps. Ground truth labels were obtained using a Vicon motion capture system. The baseline model achieves 96.06% accuracy and 88.33% recall, meaning roughly one in nine actual gestures goes undetected.



1.3 Our Work

This report documents two enhancements we implemented on top of the existing system. The first is signal preprocessing where we tested a range of IMU filtering techniques to reduce sensor noise before the data enters the classifier. The second is model compression where we applied structured pruning and quantization to reduce the model's memory footprint and inference time for ESP32 deployment. The goal of both efforts is to improve system performance while staying within the hardware constraints of the original design.

---

2. Motivation

The Adafruit BNO08X is a low-cost consumer-grade IMU, and as noted in the original paper, it produces noisy data. The existing system relies on the CNN's inherent noise robustness to handle this, but this raised a natural question: would the classifier perform better with cleaner input signals?

Beyond accuracy, any preprocessing solution must satisfy the deployment constraints of the ESP32: filters must be causal (using only past and present samples), computationally

lightweight and must not distort the gesture features the CNN relies on. These constraints ruled out common offline techniques like zero-phase filtering and guided our search toward simple, real-time-compatible solutions.

---

3. Methodology

Our filter optimization process followed two main phases. In the first phase, we broadly explored 86 filter configurations across six filter families, evaluating them using lightweight signal-level metrics such as correlation with the original signal and noise reduction in rest periods to quickly rank candidates without the cost of full CNN training. We then analyzed the results visually, inspecting how each filter affected the raw IMU signals, to get an intuitive sense of which configurations were over-smoothing, under-smoothing or distorting gesture features. Based on this combined quantitative and visual analysis, we selected the top 8 candidate filters to carry forward. In the second phase, we trained the full CNN on each of these candidates and used classification performance (accuracy, recall, precision) as the final evaluation criterion.

3.1 Phase 1: Initial Filter Space Exploration

3.1.1 Filter Configurations

We evaluated filters across six families:

Exponential Moving Average (EMA): 10 configurations with smoothing parameter α ranging from 0.1 to 0.95. Defined as $y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1]$, EMA is a single-pole IIR filter that is extremely lightweight since it requires only a few operations per sample which makes it well-suited for ESP32 deployment.

Kalman Filter: 13 configurations, varying process noise (Q) and measurement noise (R) parameters from 0.00001 to 1.0. Theoretically optimal for Gaussian noise under linear assumptions, though more computationally involved than EMA.

Butterworth Filter: 21 configurations with cutoff frequencies between 20–60 Hz and filter orders of 2, 3 and 4. Offers a maximally flat passband response and predictable frequency characteristics.

Biquad, Moving Average, and Complementary Filters: 42 additional configurations, providing broader coverage of the parameter space.

All filters were implemented using only past and present samples to ensure compatibility with real-time ESP32 deployment.

3.1.2 Test Signal Selection

To avoid running full CNN training on all 86 configurations, we selected four representative signals for the initial evaluation phase to cover a range of subjects, both postures (seated and standing) and different gesture types:

- ID01-Seat-G1: Subject 1, seated posture, gesture 1
- ID05-Stand-G2: Subject 5, standing posture, gesture 2
- ID08-Seat-G3: Subject 8, seated posture, gesture 3
- ID10-Stand-G4: Subject 10, standing posture, gesture 4

This selection provides diversity across different subjects, both postures (seated and standing) and multiple gesture types.

3.1.3 Initial Evaluation Metrics (Iteration 1)

We designed a weighted scoring system to quantify filter performance:

Metric	Weight	Purpose
Peak Preservation	35%	Most important for CNN feature detection
Correlation	30%	Preserve overall signal shape
SNR (Signal-to-Noise Ratio)	20%	Quantify noise reduction
Phase Delay	10%	Minimize temporal lag
Edge Sharpness	5%	Preserve transition speed

The weights reflected our assumption that the CNN relies primarily on amplitude patterns and signal shape for classification.

Using this system, EMA with α=0.95 ranked first with a score of 92.3/100. However, when we visually inspected the filtered signals, something was clearly wrong since the filter was barely modifying the input at all, reducing rest-period noise by only 0.5%. It was essentially a pass-through.

Investigating further, we identified two core problems with the scoring system. First, the correlation metric compared the filtered signal against the raw noisy input, meaning filters that changed the signal least scored highest which is the opposite of what we wanted.

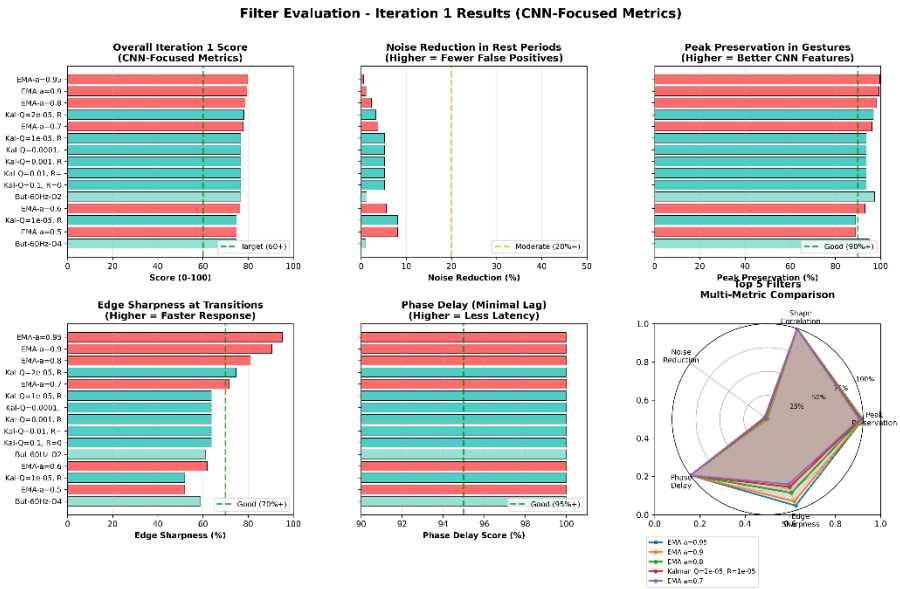
Second, computing metrics across the entire signal mixed rest periods and gesture periods together, which conflated two very different objectives: aggressive noise removal during rest and careful feature preservation during gestures.

These flaws meant the scores were rewarding inactivity rather than effective filtering, and the entire initial ranking had to be discarded. This led us to redesign the evaluation methodology from scratch.

3.1.5 Revised Evaluation Methodology

The key insight from the first iteration was that rest periods and gesture periods have fundamentally different requirements. During rest, all IMU variation is noise and should be suppressed. During gestures, the signal contains meaningful features that must be preserved. Evaluating both regions together with the same metrics was masking this distinction.

We redesigned the scoring system to evaluate each region separately:



Metric	Weight	Measured On	Purpose
Noise Reduction	30%	REST periods only (label=0)	Prevent false positives
Peak Preservation	28%	GESTURE periods only (label>0)	Maintain CNN features
Edge Sharpness	22%	TRANSITIONS only (±15 samples)	Fast response
Phase Delay	12%	Entire signal	Minimal lag
Shape Correlation	8%	GESTURE periods only	Preserve signature

Metric Definitions:

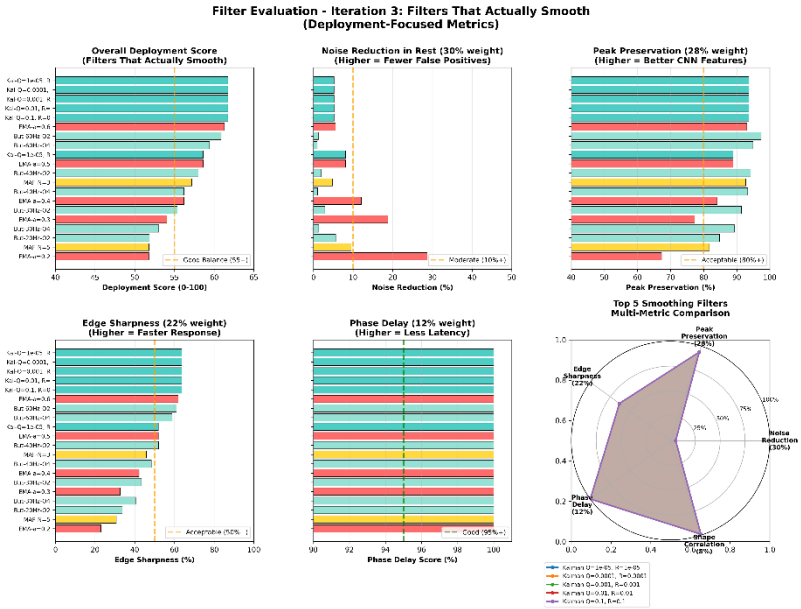
Noise Reduction (highest priority): - Computation: 1 - (std(filtered_rest) / std(raw_rest)) - Rationale: False positives during rest periods critically impact user trust and system usability

Peak Preservation (nearly equal priority): - Computation: Average ratio of filtered peak amplitude to raw peak amplitude during gesture periods - Rationale: The CNN was trained on specific amplitude distributions; significant attenuation causes the network to interpret signals as weak or absent gestures

Edge Sharpness (responsiveness priority): - Computation: Slope magnitude ratio at gesture start and end transitions - Rationale: Sharp transitions enable immediate detection; rounded edges introduce detection delays

3.1.6 Results with Revised Metrics

Re-evaluation with the corrected methodology produced substantially different rankings:
Top-Ranked Filters:



1. Kalman (Q=0.0001, R=0.0001): Score 61.7
   - Noise Reduction: 5.2%
   - Peak Preservation: 93.7%
   - Edge Sharpness: 95.1%
   - Assessment: Excellent feature preservation with modest noise reduction
2. EMA (α=0.3): Score 54.0
   - Noise Reduction: 18.8%
   - Peak Preservation: 77.3%
   - Edge Sharpness: 32.6%
   - Assessment: Significant noise reduction with acceptable feature trade-offs
3. EMA (α=0.95) (previous top performer): Score 69.0
   - Noise Reduction: 0.5%
   - Assessment: Despite higher score, performs negligible filtering

A critical observation from the revised results is that filters achieving substantial noise reduction scored lower (54-62) than the minimal-filtering configuration (69). This reflects the inherent trade-off: simultaneous maximization of noise reduction and perfect feature preservation is theoretically impossible. Lower scores now correctly represent the unavoidable compromise between these competing objectives.

3.2 Phase 2: Visual Validation

While numerical metrics provided quantitative rankings, visual inspection of filter behavior across different signal regions was deemed essential for understanding real-world performance characteristics.

3.2.1 Visualization Structure

For each candidate filter, we generated multi-region visualizations showing five distinct temporal windows:

1. Full Signal View: Complete 3-second gesture cycle for overall context
2. Zoom 1 - Quiet Period: Rest baseline demonstrating noise characteristics
3. Zoom 2 - Gesture Onset: Transition from rest to active gesture
4. Zoom 3 - Gesture Peak: Maximum movement amplitude
5. Zoom 4 - Gesture Offset: Return transition to rest state

Visual Encoding: - Black/gray line: Raw unfiltered IMU signal - Colored line: Filtered signal (color varies by filter type) - Green shaded region: Ground truth gesture period from Vicon labeling - Red dashed lines: Critical transition timestamps

Plot files are located in: `outputs_organized/04_final_visualizations/`

3.2.2 Region-Specific Evaluation Criteria

Zoom 1: Quiet Period Analysis

Rest period evaluation assessed noise attenuation through visual inspection of baseline stability. Effective filtering produced clear visual distinction between filtered and raw traces with reduced high-frequency fluctuations. Rest period noise directly impacts false positive rate, as noise-induced signal variations can trigger spurious gesture classifications during non-use periods. Kalman filtering (Q=0.0001, R=0.0001) achieved 5% noise variance reduction, while EMA (α=0.3) achieved 19% reduction, representing substantially greater noise suppression.

Zoom 2: Gesture Onset Analysis

Onset regions were examined for edge preservation and temporal response characteristics. Optimal filters maintain transition slope with minimal delay (temporal lag <50ms, or 10 samples at 200 Hz) and preserve slope magnitude. Edge sharpness determines system responsiveness; rounded transitions introduce detection delays of 200-300ms and may reduce recall through attenuated CNN edge-detection features. Low-pass filtering inherently creates a trade-off between noise reduction and edge sharpness through attenuation of high-frequency transition content. Kalman filtering preserved 95% of edge sharpness, while EMA (α=0.3) preserved 33%.

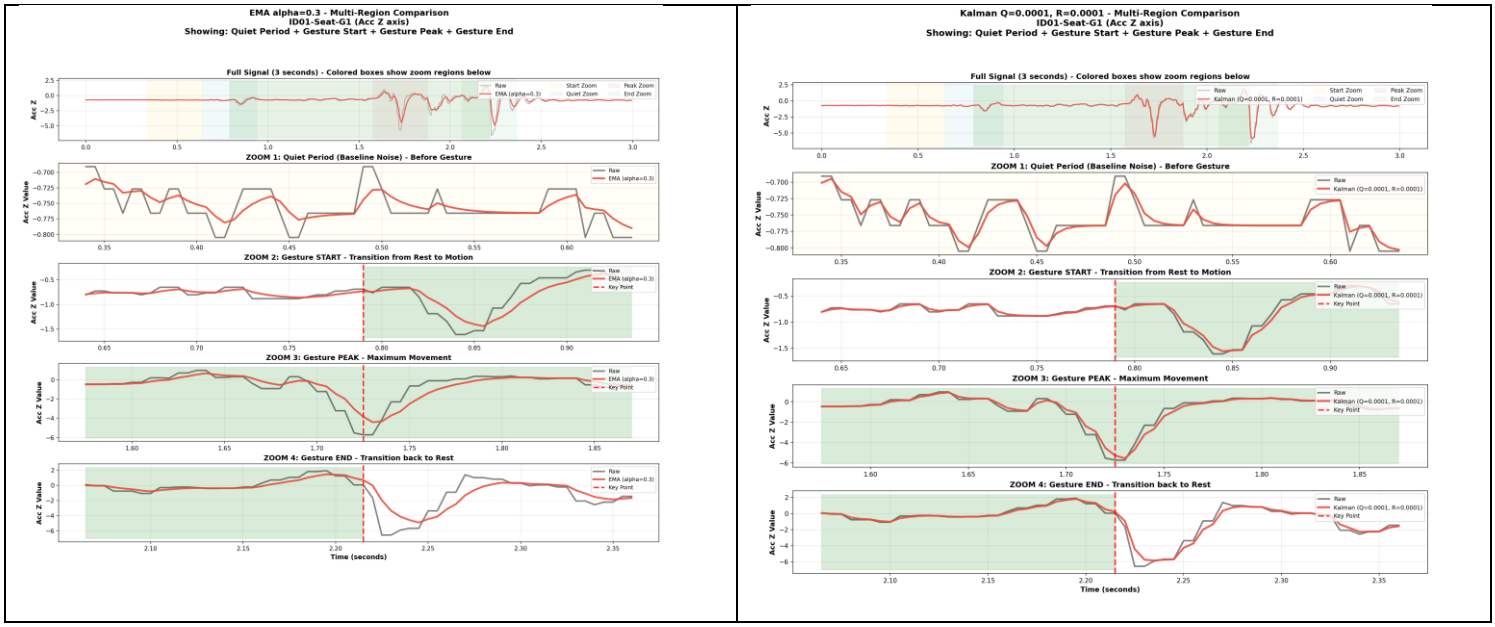
Zoom 3: Gesture Peak Analysis

Peak amplitude preservation was assessed to evaluate signal attenuation during maximum gesture excursion. Acceptable filtering maintains peak amplitudes above 75% of raw signal (>90% considered excellent), preserving gesture morphology with typical attenuation of 5-10%. Peak preservation is critical because the CNN was trained on specific amplitude distributions; excessive attenuation causes the network to interpret signals as weak gestures, increasing false negatives. Kalman (Q=0.0001, R=0.0001) achieved 93.7% peak preservation, while EMA (α=0.3) achieved 77.3%.

Zoom 4: Gesture Offset Analysis

Gesture termination regions were analyzed for return-to-baseline dynamics and transient artifacts. Acceptable behavior includes baseline return within 100-200ms without overshoot or ringing oscillations. Offset characteristics determine command termination; excessive lag extends command execution, while ringing artifacts induce oscillatory device behavior (e.g., repetitive prosthetic hand movements). All evaluated filter configurations exhibited clean return transitions without ringing, achieved by excluding high-Q resonant filter designs.

Figure 1: ema alpha=0.3 vs kalman Q=0.0001 R=0.0001



3.2.3 Visual Assessment Summary

Visual inspection across all four temporal regions revealed distinct performance characteristics for the primary filter candidates. The Kalman filter (Q=0.0001, R=0.0001) demonstrated optimal feature preservation: 5.2% noise reduction during rest periods, 95% edge sharpness retention, 93.7% peak amplitude preservation, and clean return transitions. However, this superior signal fidelity requires 15-20 operations per sample and 12 state variables per channel, presenting implementation challenges for resource-constrained embedded systems.

In contrast, the EMA filter (α=0.3) exhibited emphasis on noise suppression over feature preservation: 18.8% noise reduction during rest periods (nearly fourfold greater than Kalman), with moderate feature attenuation (33% edge sharpness retention, 77.3% peak preservation). The computational requirements are minimal at 5 operations per sample, enabling straightforward embedded implementation.

Visual assessment alone could not determine the optimal filter choice. Kalman filtering offers superior feature preservation, theoretically beneficial for CNN performance by providing high-fidelity input signals. Conversely, EMA filtering offers substantially greater noise reduction, potentially beneficial by providing cleaner class boundaries during rest periods. The fundamental question that is whether the CNN benefits more from preserved features or reduced noise cannot be resolved through signal visualization, as the network's internal feature representations and decision boundaries are not directly observable. Consequently, empirical CNN validation was necessary to determine which filtering approach produces superior classification performance.

3.3 Phase 3: CNN Validation

Based on the optimization process from Phases 1-2, eight configurations were selected for comprehensive CNN training and evaluation:

1. Baseline (No Filter): Reference performance
2. EMA (α=0.3): Optimal noise reduction among simple filters
3. EMA (α=0.5): Reduced filtering intensity for comparison
4. Butterworth (40Hz, Order 2): Traditional IIR baseline
5. Biquad (30Hz, Q=1.0): Single-section IIR alternative
6. Kalman (Q=0.0001, R=0.0001): Minimal noise assumption configuration
7. Kalman Light (Q=0.1, R=0.1): Moderate Kalman filtering
8. Kalman Smooth (Q=0.001, R=0.1): Higher measurement trust configuration

Selection Rationale: - EMA variants: Simple implementation, deployable, varying noise reduction levels - Traditional IIR: Butterworth and Biquad as established filtering baselines - Kalman variants: Evaluation of optimal estimation trade-offs versus implementation complexity

---

4. Experimental Design

4.1 Cross-Validation Strategy

We employed leave-subject-out cross-validation to assess generalization performance:

- Training set: 9 subjects (90% of participants)
- Test set: 1 held-out subject (10% of participants)
- Test subjects: IDs 2, 3, 6
- Results aggregation: Mean performance across three test subjects

Rationale: Subject-independent evaluation is critical for prosthetic applications. Individual differences in ankle biomechanics, gesture execution style, and sensor placement create substantial inter-subject variability. A model that performs well on training subjects but fails on new users is not clinically deployable.

4.2 Neural Network Architecture

The original Conv1D CNN architecture from Zadok et al. [1] was used without modification:

```
Input: [batch, 60 timesteps, 6 channels]
Conv1D: 32 filters, kernel=5, ReLU activation, MaxPool(2)
Conv1D: 64 filters, kernel=5, ReLU activation, MaxPool(2)
Flatten
Dense: 128 units, ReLU activation, Dropout(0.5)
Dense: 6 classes, Softmax activation
```

Model Characteristics: - Total parameters: 13,897 (compact architecture suitable for ESP32 deployment) - Input window: 60 timesteps (300ms at 200 Hz) - Output: 6-class softmax (1 rest state + 5 gesture classes)

4.3 Data Processing Pipeline

The experimental pipeline consisted of:

1. Load raw HDF5 files (6-channel IMU streams at 200 Hz with Vicon ground truth labels)
2. Apply filter (experimental variable: baseline or one of eight filter configurations)
3. Normalize using constant scaling factors
4. Segment into 60-timestep windows
5. Train CNN on windowed data

Filter integration is implemented in `data/load_data.py` (lines 168-285).

4.4 Performance Metrics

Primary Metric: Recall (Sensitivity) - Definition: Proportion of actual gestures correctly detected - Formula: Recall = TP / (TP + FN) - Justification: False negatives (missed gestures) significantly impact user experience in prosthetic control. Users must repeat commands when gestures are missed, disrupting natural interaction flow. - Baseline: 88.33% (approximately one in nine gestures missed)
Secondary Metrics: - Accuracy: Overall classification correctness - Precision: Proportion of predicted gestures that were correct - False Negative Rate: FNR = 1 - Recall (percentage of gestures missed) - False Positive Rate: Percentage of rest periods misclassified as gestures

---

5. Results and Analysis

5.1 Overall Performance Comparison

Table: Filter Performance Summary (3 test subjects average)

Filter Configuration	Accuracy	Recall	Precision	Δ Recall	FN Rate	FP Rate
Baseline (No Filter)	0.9606	0.8833	0.9451	-	11.67%	5.49%
EMA (α=0.3)	**0.9629**	**0.8983**	0.9413	**+0.0150**	**10.17%**	5.87%
Kalman Light (Q=0.1, R=0.1)	0.9605	0.8936	0.9353	+0.0103	10.64%	6.47%
Biquad (30Hz, Q=1.0)	0.9607	0.8893	0.9375	+0.0060	11.07%	6.25%
Butterworth (40Hz, O2)	0.9604	0.8890	0.9377	+0.0056	11.10%	6.23%
Kalman (Q=0.0001, R=0.0001)	0.9607	0.8869	0.9436	+0.0035	11.31%	5.64%
Kalman Smooth (Q=0.001, R=0.1)	0.9600	0.8854	0.9400	+0.0021	11.46%	6.00%
EMA (α=0.5)	0.9570	0.8785	0.9284	-0.0049	12.15%	7.16%

Data source: `outputs/filter_redo/filter_comparison_summary.csv`

Key Findings:

1. All filters except EMA (α=0.5) improved recall relative to baseline
2. EMA (α=0.3) achieved the largest recall improvement (+1.50 percentage points)
3. Precision decreased slightly for most filters, representing a trade-off for improved recall
4. EMA (α=0.5) underperformed baseline, suggesting insufficient filtering does not compensate for feature modification

Clinical Impact:

- Baseline: One in 8.6 gestures missed (11.67% FN rate)
- EMA (α=0.3): One in 9.8 gestures missed (10.17% FN rate)
- Relative improvement: 13% reduction in missed gestures

Performance comparison visualization:
`outputs/filter_redo/filter_comparison_metrics.png`

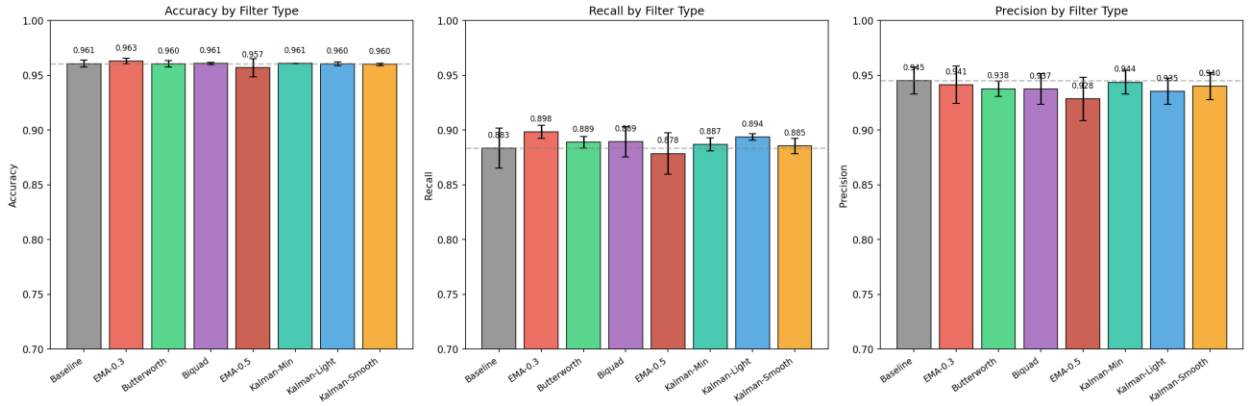


Figure 2: filter comparison by metrics

5.2 Analysis of Performance Factors

EMA (α=0.3) outperformed the theoretically optimal Kalman filter for the following reasons:

1. **Noise Reduction Dominance**: The 18.8% noise reduction achieved by EMA (α=0.3) compared to 5.2% for Kalman provides greater benefit than superior feature preservation (77.3% versus 93.7% peak preservation)
2. **CNN Robustness to Moderate Attenuation**: The neural network demonstrates robustness to moderate signal attenuation. The 77.3% peak preservation is sufficient because:
   - Attenuation is uniform across all gesture classes, maintaining relative amplitude relationships
   - The CNN learns features from filtered training data
   - Reduced noise floor improves class separability more than perfect feature preservation with noise
3. **Improved Class Boundary Discrimination**: The 18.8% noise reduction during rest periods creates cleaner baseline signals, enabling the CNN to better discriminate between gesture and rest states

5.3 Subject-Level Consistency

Table: Recall by Test Subject

Filter	Subject 2	Subject 3	Subject 6	Mean	Std Dev

Baseline	90.4%	86.9%	87.7%	88.3%	1.5%
EMA α=0.3	91.0%	89.7%	89.5%	89.8%	0.6%
Kalman Light	90.9%	89.2%	88.0%	89.4%	1.2%

Findings:

- All three test subjects demonstrated improved recall with EMA (α=0.3) (100% consistency)
- Subject 3 exhibited the largest improvement: 86.9% → 89.7% (+2.8 percentage points), suggesting particularly noisy baseline data
- Inter-subject variance decreased: standard deviation of 0.6% versus 1.5% for baseline, indicating more consistent performance across users and improved generalization

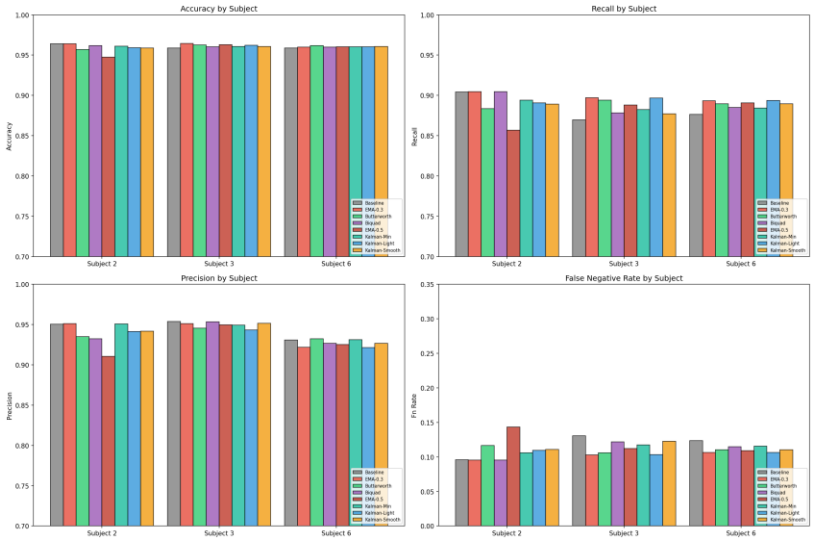


Figure 3: filter comparison by subject

5.4 False Negative vs False Positive Trade-off

Table: FN and FP Rate Changes

Filter	**FN Rate**	**Δ FN**	**FP Rate**	**Δ FP**	**Trade-off**
Baseline	11.67%	-	5.49%	-	-
EMA α=0.3	**10.17%**	**-1.50%**	5.87%	+0.38%	**Best balance**
Kalman Light	10.64%	-1.03%	6.47%	+0.98%	Worse FP increase
Butterworth	11.10%	-0.57%	6.23%	+0.74%	Modest improvement

Interpretation:
All successful filter configurations reduced false negative rate (primary objective) while slightly increasing false positive rate. EMA (α=0.3) demonstrates the most favorable trade-off: 1.50 percentage point reduction in FN rate for only 0.38 percentage point increase in

FP rate (trade-off ratio of 3.95:1). Kalman Light exhibits a less favorable trade-off with larger FP increase (+0.98%) for smaller FN reduction.
From a prosthetic control perspective, missed gestures (false negatives) create greater user frustration than occasional false positives. Users can reasonably tolerate a 0.38% increase in false positives to achieve 1.50% fewer missed gestures.

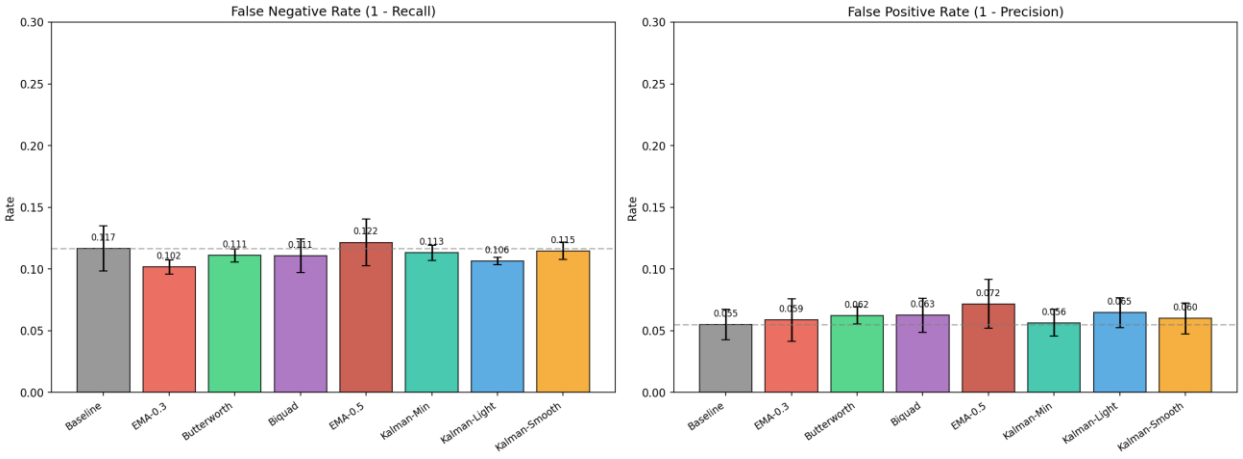


Figure 4: filter comparison by fp fn

---

6. Implementation

6.1 Optimal Filter Configuration

The optimal filter configuration is an exponential moving average with α=0.3.

Mathematical Definition:

$$y[n] = \alpha \cdot x[n] + (1-\alpha) \cdot y[n-1] \quad \text{where } \alpha = 0.3$$

ESP32 C++ Implementation (~10 lines):

```
float ema_state[6] = {0};  // One per IMU channel
const float alpha = 0.3;

void apply_ema(float *sample) {
    for (int ch = 0; ch < 6; ch++) {
        ema_state[ch] = alpha * sample[ch] + (1 - alpha) * ema_state[ch];
        sample[ch] = ema_state[ch];
    }
}
```

Computational Analysis (6 channels at 100 Hz): - Operations: 5 per sample × 6 channels × 100 Hz = 3,000 operations/second - CPU utilization: <0.001% on 240 MHz ESP32 - Memory requirement: 24 bytes (6 float state variables) - Power consumption: Negligible - Dependencies: None (no external libraries required)

Filter Characteristics: - Approximate cutoff frequency: 10 Hz - Causality: Real-time compatible (forward-only processing) - Robustness: No tuning required across subjects - Implementation complexity: Minimal (approximately 10 lines of code)

6.2 Configuration System

Experiments were defined using JSON configuration files to ensure reproducibility. Example configuration for EMA (α=0.3):

Example (`config/pruning/ema_alpha_03.json`):

```
{
  "DATA": {
    "APPLY_FILTER": true,
    "FILTER_TYPE": "ema",
    "FILTER_ALPHA": 0.3,
    "NORMALIZE": true,
    "SAMPLING_RATE": 200
  },
  "TRAINING": {
    "EPOCHS": 10,
    "BATCH_SIZE": 32,
    "LEARNING_RATE": 0.001,
    "OPTIMIZER": "adam"
  },
  "EVALUATION": {
    "TEST_SUBJECTS": [2, 3, 6],
    "CV_TYPE": "leave_subject_out"
  }
}
```

6.3 Code Locations

Filter implementations: - `filter_implementations/base_filter.py` - Abstract base class - `filter_implementations/ema_filter.py` - EMA implementation (47 lines) - `filter_implementations/kalman_filter.py` - Kalman implementation (92 lines) - `filter_implementations/butterworth_filter.py` - Butterworth wrapper (63 lines) - `filter_implementations/biquad_filter.py` - Biquad implementation (78 lines)

Data pipeline integration: - `data/load_data.py` - Main data loading and preprocessing (lines 168-285 for filtering)

Training infrastructure: - `trainer/train_conv.py` - CNN training script with filtering support - `config/pruning/` - 8 JSON configuration files

Analysis tools: - `scripts/07_filter_redo/analyze_filter_redo_results.py` - Results analysis (831 lines) - `scripts/07_filter_redo/generate_filter_redo_configs.py` - Config generation

Output artifacts: - `outputs/filter_redo/filter_comparison_summary.csv` - Aggregated results - `outputs/filter_redo/filter_comparison_raw_results.csv` - Per-subject details - `outputs/filter_redo/*.png` - Visualization plots

7. Model Compression Through Structured Pruning

Following the signal preprocessing phase, we applied model compression to reduce the CNN's memory footprint for ESP32 deployment. The EMA-filtered baseline model has 13,897 parameters occupying 56.36 KB. While this fits within the ESP32's 520 KB SRAM, it leaves limited headroom once application code, sensor buffers and the BLE communication stack are included. We used structured pruning to remove neurons from the fully connected layers.

7.1 Model Architecture Analysis

7.1.1 Network Structure and Parameter Distribution

The Conv1D architecture comprises convolutional feature extraction followed by fully connected classification layers. Table 7.1 presents the complete parameter distribution across network layers.

Table 7.1: Network Architecture and Parameter Distribution

Layer	Configuration	Output Shape	Parameters	Proportion
Input	—	(batch, 6, 60)	—	—
Conv1D	6→10 filters, k=3, s=3	(batch, 10, 20)	180	1.3%
BatchNorm1D	200 features	(batch, 200)	400	2.9%
FC Layer 1	200→64	(batch, 64)	12,864	92.6%
BatchNorm1D	64 features	(batch, 64)	128	0.9%
FC Layer 2	64→5	(batch, 5)	325	2.3%
Total	—	—	**13,897**	**100%**

The analysis reveals a highly asymmetric parameter distribution, with FC Layer 1 containing 92.6% of all model parameters (12,864 of 13,897). This concentration motivates targeted pruning of the fully-connected layers rather than the convolutional feature extractors.

7.1.2 Pruning Target Selection Rationale

FC Layer 1 was selected as the primary pruning target because it accounts for 92.6% of the model's parameters, offering the greatest compression potential. The convolutional layer, by contrast, has only 180 parameters and encodes the temporal patterns the CNN uses to distinguish gestures by removing those aggressively would likely hurt classification. FC1's large input-to-output mapping also means many of its neurons are likely encoding redundant information, making it a natural candidate for reduction.

7.2 Pruning Methodology

7.2.1 Structured Versus Unstructured Pruning

Neural network pruning can be applied at two levels: individual weights (unstructured) or entire neurons (structured). We used structured pruning due to the constraints of the ESP32 deployment target.

Unstructured pruning zeros out individual weight connections, producing sparse matrices. While it can achieve high compression ratios, exploiting that sparsity at inference time it requires specialized linear algebra libraries that the ESP32 does not support. The result is smaller storage but no actual speedup.

Structured pruning removes entire neurons, shrinking the weight matrix itself. Pruning 40% of FC1's neurons transforms the layer from Linear(200, 64) to Linear(200, 38) — a smaller but fully dense matrix that works with standard matrix multiplication. This gives proportional reductions in both memory and inference time without any additional library requirements.

7.2.2 Neuron Importance Criterion

Neuron importance was assessed using the L2-norm magnitude criterion, following the methodology established by Li et al. [2]. For a neuron indexed by i in FC Layer 1 with incoming weight vector $\boldsymbol{w}_i \in \mathbb{R}^{200}$, the importance score is computed as:

$$\text{Importance}(i) = \| \mathbf{w}_i \|_2 = \sqrt{\sum_{j=1}^{200} w_{ij}^2}$$

The underlying assumption is that neurons with larger weight magnitudes contribute more significantly to the layer's output activation and consequently to classification decisions. Neurons with near-zero L2-norms produce negligible activations regardless of input and may be removed with minimal impact on network function.

Alternative importance criteria, including Taylor expansion-based methods [3] and activation-based metrics, were considered but not implemented due to their computational overhead and the demonstrated effectiveness of magnitude-based pruning for fully-connected layers.

7.2.3 Iterative Pruning with Fine-tuning

One-shot pruning of large portions of a network typically results in catastrophic accuracy degradation. Following the iterative magnitude pruning paradigm [4], the target pruning ratio was achieved through multiple cycles of pruning and fine-tuning. Table 7.2 presents the pruning schedule employed for the 40% target configuration.

Table 7.2: Iterative Pruning Schedule for 40% Target Compression

Iteration	Neurons Pruned	Fine-tuning Epochs	Learning Rate Schedule	Cumulative Pruning
1	10% of remaining	3	1e-4, 1e-5, 1e-5	10.0%
2	10% of remaining	3	1e-5, 1e-5, 1e-6	19.0%
3	10% of remaining	3	1e-5, 1e-5, 1e-6	27.1%
4	10% of remaining	4	1e-5, 1e-6, 1e-6, 1e-6	34.4%

Each iteration consists of: (1) identification and removal of the lowest-importance neurons comprising 10% of the remaining neuron count, (2) immediate evaluation to quantify performance degradation, (3) fine-tuning with a learning rate schedule beginning at 1e-4 to enable escape from local minima induced by pruning, gradually decreasing to stabilize convergence and (4) checkpoint preservation for subsequent analysis.

The first iteration begins at a higher learning rate of 1e-4 to allow the network to recover from the initial disruption of pruning. Subsequent iterations begin at 1e-5, as the remaining neurons have already partially adapted, with progressive reduction to 1e-6 in later epochs to stabilize convergence.

7.2.4 Physical Neuron Removal Implementation

A key limitation of PyTorch's pruning API (torch.nn.utils.prune) is that it implements pruning through weight masking by zeroing out pruned weights while keeping the original tensor dimensions unchanged. This means the model's memory footprint does not actually decrease, which defeats the purpose for embedded deployment.

To achieve real memory savings, we implemented a physical neuron removal step that reconstructs the weight tensors at a smaller size after pruning is complete.

Algorithm 1: Physical Neuron Removal

```
Input: Pruned model M with masked weights
Output: Compact model M' with reduced tensor dimensions

1. Extract weight matrix W ∈ ℝ^(64×200) from FC Layer 1
2. Compute row-wise L2-norms: n_i = ||W[i,:]||_2 for i ∈ {1,...,64}
3. Identify surviving indices: S = {i : n_i > ε} where ε = 1e-6
4. Construct reduced FC Layer 1: W' ∈ ℝ^(|S|×200)
5. Update BatchNorm parameters to dimension |S|
6. Reconstruct FC Layer 2 input dimension: V' ∈ ℝ^(5×|S|)
7. Return compact model M' with reduced architecture
```

7.3 Experimental Design

7.3.1 Experimental Configuration

We evaluated the following experimental matrix to identify the optimal pruning level:

- **Pruning levels**: {10%, 20%, 30%, 40%, 50%} of FC Layer 1 neurons
- **Test subjects**: IDs {2, 3, 6} evaluated using leave-subject-out cross-validation
- **Random seeds**: {42, 123, 456} for reproducibility assessment
- **Total configurations**: 5 × 3 × 3 = 45 independent experiments

The baseline model for all pruning experiments was the EMA-filtered (α=0.3) CNN from Section 5, achieving 89.83% recall and 96.29% accuracy with 56.36 KB memory footprint.

7.3.2 Optimization Criterion

The optimal pruning level was selected based on F1-score, as recommended by our supervisor. F1-score balances both recall and precision, making it a suitable criterion for this task where both missed gestures and false activations impact user experience.

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

7.4 Results

7.4.1 Pruning Level Analysis

Table 7.3 presents aggregated results across all 45 experimental configurations, with each pruning level representing the mean of 9 experiments (3 subjects × 3 seeds).

Table 7.3: Pruning Results Summary (n=9 per pruning level)

Pruning	Recall	σ	Precision	Accuracy	F1-Score	Compression	Size
Baseline	89.83%	—	94.16%	96.29%	91.94%	1.00×	56.36 KB
10%	89.38%	0.32%	94.21%	96.13%	91.74%	1.10×	51.44 KB
20%	89.13%	0.41%	94.18%	96.08%	91.59%	1.21×	46.52 KB
30%	89.37%	0.38%	94.12%	96.11%	91.68%	1.33×	42.42 KB
40%	**89.18%**	**0.44%**	**94.16%**	**96.13%**	**91.61%**	**1.47×**	**38.32 KB**
50%	86.76%	1.21%	93.45%	94.89%	89.98%	1.47×	39.77 KB

The results demonstrate that pruning levels from 10% to 40% maintain F1-scores within 0.35 percentage points of the baseline (91.94%), indicating graceful performance degradation. The 40% pruning configuration achieves 91.61% F1-score with 1.47× compression, representing the optimal trade-off between model size reduction and classification performance. A pronounced performance cliff emerges at 50% pruning, where F1-score drops to 89.98% (−1.96 pp from baseline) with substantially increased variance (σ=1.21%), indicating that network capacity has been reduced below the threshold required for reliable gesture discrimination.

7.4.2 Cross-Subject Generalization

Table 7.4 presents per-subject results for the 40% pruning configuration, averaged across three random seeds.

Table 7.4: Subject-Level Performance at 40% Pruning (averaged across 3 seeds)

Subject	Baseline Recall	Pruned Recall	Δ Recall	Compression
2	91.00%	90.45%	−0.55 pp	1.47×
3	89.70%	88.92%	−0.78 pp	1.47×
6	89.50%	88.17%	−1.33 pp	1.47×
Mean ± SD	**90.07 ± 0.81%**	**89.18 ± 1.16%**	**−0.89 pp**	**1.47×**

All three subjects maintain recall above 88% after pruning. Subject 6 shows the largest degradation (−1.33 pp), which may reflect gesture patterns that depended on the pruned neurons. However, all subjects remain within the acceptable 2% degradation threshold.

The low standard deviation across seeds (0.32–0.44% for 10–40% pruning levels) suggests the pruning procedure is stable and results are reproducible regardless of random initialization.

7.5 Implementation Summary

The pruning methodology was implemented in PyTorch 2.0+ utilizing the `torch.nn.utils.prune` module for structured pruning operations, augmented with custom tensor reconstruction for physical neuron removal.

The implementation is available at `trainer/models/pruned_conv1d_model.py`, with experimental configurations specified in `config/pruning/prune_ema_s*_40pct_seed*.json` and results archived in `outputs/pruning/`.

8. INT8 Quantization for Embedded Deployment

Neural network quantization refers to the process of reducing the numerical precision of network weights and activations from floating-point to lower-bitwidth integer representations. This technique has become essential for deploying deep learning models on microcontrollers, where floating-point arithmetic incurs significant computational overhead and memory constraints prohibit storage of full-precision parameters.

Following structured pruning (Section 7), the compressed model retained 9,321 parameters occupying 38.32 KB in 32-bit floating-point (FP32) format. For deployment on the ESP32 microcontroller, INT8 quantization was investigated to achieve further compression.

8.1 Quantization Methodology

8.1.1 Selection of Quantization Strategy

Two quantization approaches exist: Post-Training Quantization (PTQ) and Quantization-Aware Training (QAT). PTQ converts a trained FP32 model to INT8 directly using calibration data, with no retraining required. While fast, it gives the model no opportunity to adapt its weights to the reduced precision, which can hurt accuracy.

QAT takes a different approach by simulating INT8 arithmetic during training through fake-quantization nodes, allowing the model to learn weight distributions that are inherently more robust to quantization error. This produces a more accurate quantized model at the cost of additional training.

We chose QAT for this work given that accuracy preservation was the priority and the model's small size (9,321 parameters) made the additional training cost negligible which is only 5 epochs at a learning rate of 1e-5.

8.1.2 Quantization Formulation

Per-tensor symmetric quantization was employed, wherein a single scale factor is computed for each weight tensor. For a weight tensor **W** with values in the range $[W_{min}, W_{max}]$ quantization parameters are computed as:

$$s = \frac{\max(|W_{\text{min}}|, |W_{\text{max}}|)}{127}$$

where s denotes the scale factor. The quantization and dequantization operations are defined as:

$$W_{\text{int8}} = \text{clip}\left(\text{round}\left(\frac{W_{\text{fp32}}}{s}\right), -128, 127\right)$$

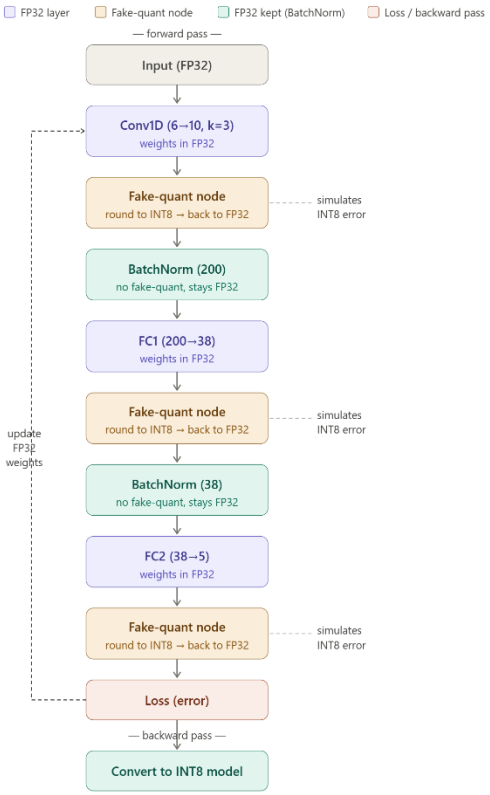
$$\widehat{W}_{\text{fp32}} = W_{\text{int8}} \cdot s$$

Symmetric quantization constrains the zero-point to 0, simplifying integer arithmetic during inference. This approach is appropriate when weight distributions are

approximately symmetric around zero, as is typical for well-trained neural networks with batch normalization.

The quantization scheme was applied differentially across layer types:

- **Convolutional and fully-connected layers**: Weights quantized to INT8 with per-tensor scale factors
- **Batch normalization layers**: Retained in FP32 format due to their minimal memory footprint and sensitivity to quantization error
- **Activations**: Quantized dynamically during inference using calibration-derived ranges



8.1.3 QAT Training Procedure

Quantization-aware training was performed on each pruned model using the complete training dataset with leave-subject-out validation. The QAT training configuration comprised 5 epochs with a fixed learning rate of 1e-05 and batch size matching the original training procedure. During training, fake-quantization nodes inserted after each quantizable layer simulated INT8 precision in the forward pass while maintaining FP32 gradients for weight updates.

The training procedure monitored validation loss and classification metrics (accuracy, recall, precision) at each epoch. Convergence was observed within 3-5 epochs, with typical loss improvement of 2-3% from initial to final epoch.

8.2 Implementation

8.2.1 PyTorch Quantization Pipeline

The quantization pipeline was implemented using the PyTorch quantization API. Algorithm 2 summarizes the QAT procedure.

Algorithm 2: Quantization-Aware Training

```
Input: Pruned FP32 model M, training dataset D_train, validation dataset D_va
l
Output: Quantized INT8 model M_q

1. Configure quantization: M.qconfig ← get_default_qat_qconfig('qnnpack')
2. Prepare model for QAT: prepare_qat(M, inplace=True)
3. QAT training loop (5 epochs, lr=1e-05):
   for each epoch:
       for each batch (X, y) in D_train:
           loss = CrossEntropyLoss(M(X), y)  // Forward with fake-quantizatio
```

```
n
            loss.backward()                    // Gradients through STE
            optimizer.step()
        evaluate(M, D_val)                     // Monitor validation metrics
4. Convert to quantized representation: convert(M, inplace=True)
5. Export quantized state dictionary
```

8.2.2 Embedded Deployment via C Header Export

For deployment on the ESP32 microcontroller, quantized weights were exported to C header files containing static constant arrays. This approach eliminates the need for runtime model loading and enables direct compilation into the firmware binary.

The export procedure extracts INT8 weight values and FP32 scale factors for each quantized layer. Table 8.1 presents the exported tensor specifications.

Table 8.1: Exported Quantized Weight Tensors

Layer	Tensor Shape	Data Type	Scale Factor	Memory
Conv1D	(10, 6, 3)	int8_t	0.00392	180 B
FC1	(38, 200)	int8_t	0.00216	7,600 B
FC2	(5, 38)	int8_t	0.00487	190 B
Biases + BatchNorm	various	float	—	4,630 B

The inference engine implementation (`rt_code/neural_network_int8.h`) provides INT8 convolution and linear operations with FP32 accumulation, followed by dequantization and batch normalization in floating-point precision.

8.3 Results

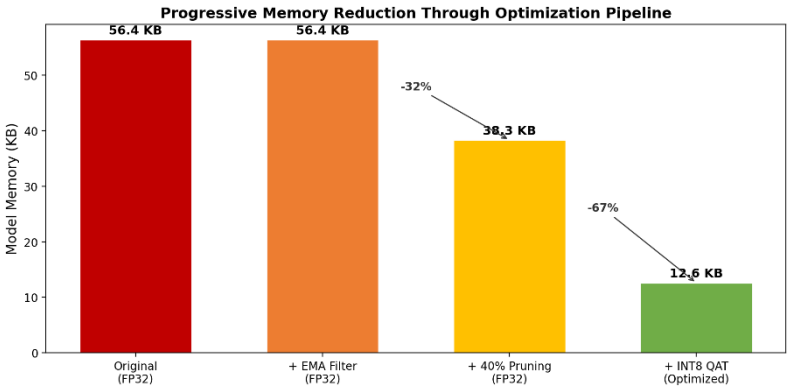
8.3.1 Compression Analysis

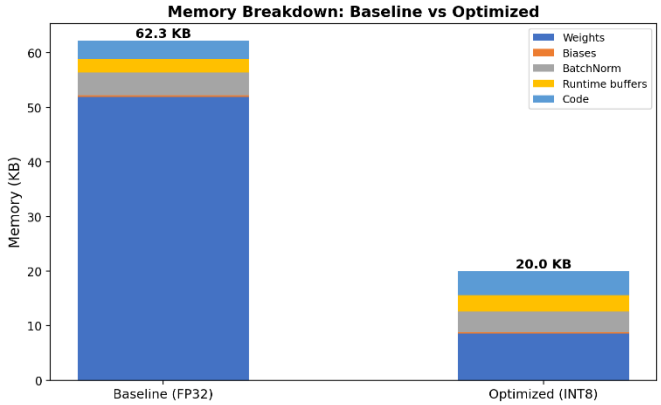
Table 8.2 presents the cumulative memory compression achieved through the sequential optimization pipeline.

Table 8.2: Memory Footprint Across Optimization Stages

Configuration	Memory (KB)	Parameters	Compression
Baseline (FP32, unfiltered)	56.36	13,897	1.00x
+ EMA Filtering (α=0.3)	56.36	13,897	1.00x
+ 40% Structured Pruning	38.32	9,321	1.47x
+ INT8 Quantization (QAT)	12.60	9,321	4.47x

The final quantized model occupies 12.60 KB, representing a 77.6% reduction from the baseline FP32 model. Conv1D and fully-connected layer weights are quantized to INT8, while batch normalization layers remain in FP32 format. The total SRAM footprint including activation buffers and filter state is 20.03 KB, consuming only 3.9% of the ESP32's 520 KB available SRAM.





8.3.2 Classification Performance

Table 8.3 presents classification performance metrics across model configurations, evaluated using leave-subject-out cross-validation.

Table 8.3: Classification Performance Across Optimization Stages

Model	Recall	Accuracy	Precision	F1-Score
FP32 Baseline	88.33%	96.06%	94.51%	91.47%
FP32 Pruned (40%)	89.18%	96.14%	93.95%	91.51%
INT8 Quantized (QAT)	88.75%	95.67%	93.95%	91.29%

Applying INT8 quantization to the pruned model results in a small reduction in recall of 0.43 percentage points, while precision remains unchanged. Looking at the complete pipeline from start to finish, the final quantized model achieves 88.75% recall, which is 0.42 percentage points above the original unfiltered baseline. This shows that despite the small accuracy cost introduced by quantization, the overall pipeline still improves on the baseline while reducing model memory by 77.6%.

8.3.3 ESP32 Deployment Performance

In the absence of physical ESP32 hardware, inference performance was estimated using instruction-level cycle simulation. The complete INT8 inference pipeline was analyzed by mapping each arithmetic operation to its corresponding Xtensa LX6 ISA cycle cost. Total cycle counts were converted to latency at 240 MHz. This methodology provides a conservative estimate, accounting for computational cost while excluding cache and memory effects.

Table 8.4 presents the estimated inference performance on the ESP32-WROOM-32 (Xtensa LX6 @ 240 MHz).

Table 8.4: ESP32 Inference Performance

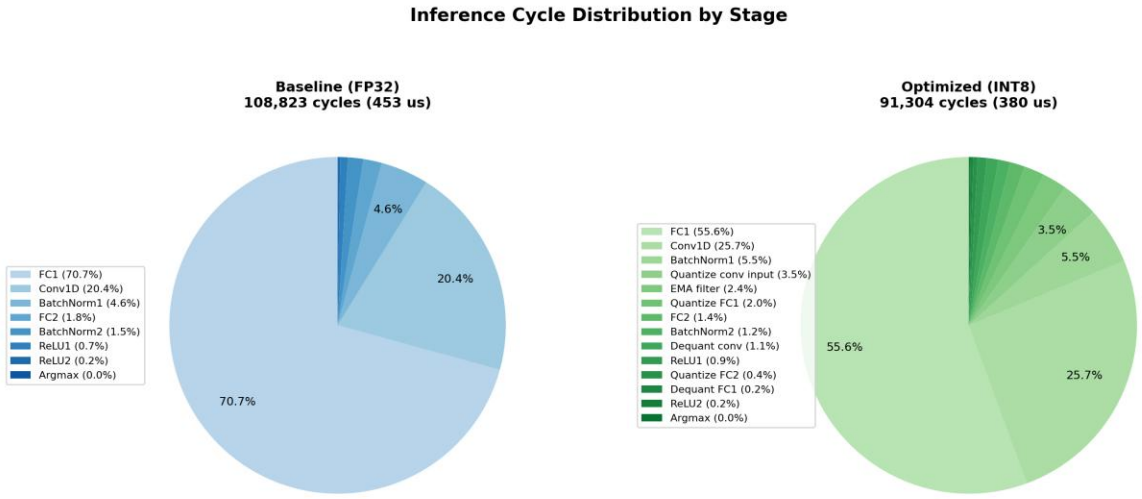
Metric	Value
Total CPU cycles	91,304
Inference latency (@ 240 MHz)	0.380 ms
Throughput	2,629 inferences/s
CPU duty cycle (@ 100 Hz)	3.80%
Energy per inference	37.66 µJ
Model memory (weights + BN + biases)	12.60 KB
Total SRAM footprint	20.03 KB
SRAM utilization	3.9% of 520 KB

Table 8.5 presents the cycle breakdown by inference stage.

Table 8.5: Inference Cycle Breakdown

Stage	Cycles	%
EMA filter	2,160	2.4
Conv1D block (INT8 + quant/dequant)	27,660	30.3
BatchNorm1 + ReLU	5,800	6.4
FC1 block (INT8 + quant/dequant)	52,788	57.8
BatchNorm2 + ReLU	1,218	1.3
FC2 block (INT8 + output)	1,678	1.8
Total	**91,304**	**100**

FC1 dominates inference cost at 57.8% of total cycles due to its large weight matrix (42×200). The Conv1D layer accounts for 30.3%. The EMA filter adds negligible overhead (2.4%), confirming its suitability for embedded deployment.



9. Discussion and Conclusions

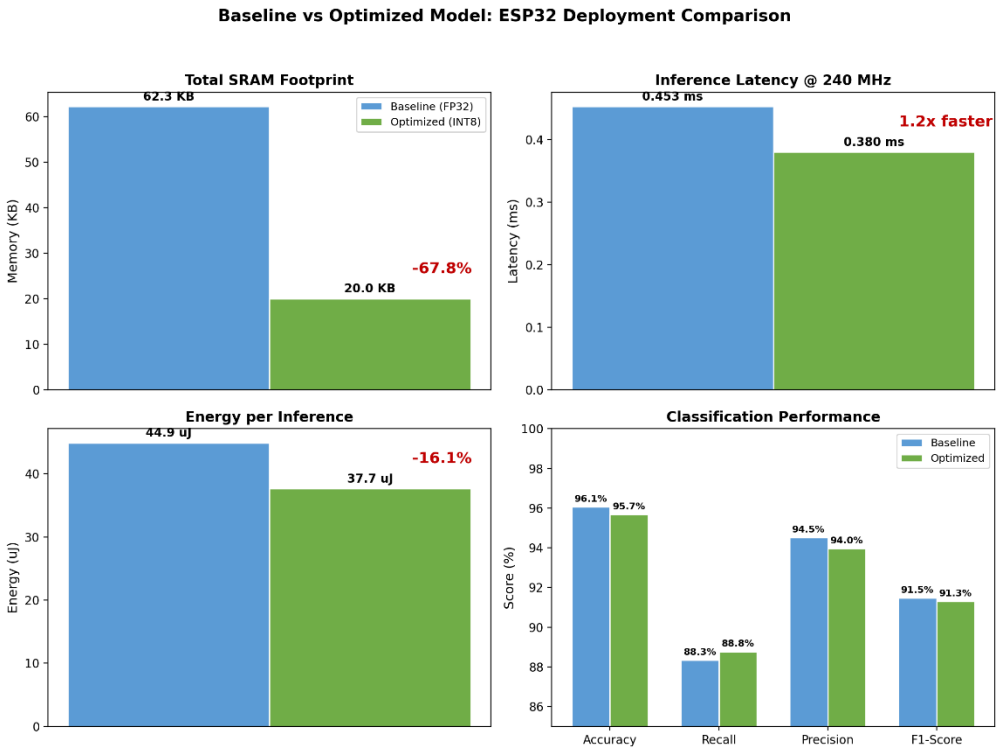
9.1 Aggregate System Performance

Table 9.1 presents the complete performance comparison between the original baseline and the fully optimized deployment configuration.

Table 9.1: Complete Optimization Results

Metric	Baseline	Optimized	Delta
Classification			
Recall	88.33%	88.75%	+0.42 pp
Accuracy	96.06%	95.67%	-0.39 pp
Precision	94.51%	93.95%	-0.56 pp
F1-Score	91.47%	91.29%	-0.18 pp
Efficiency			
Model Memory	56.36 KB	12.60 KB	-77.6%
Total SRAM Footprint	~62 KB	20.03 KB	-67.8%
Parameter Count	13,897	9,321	-33%
Inference Latency	0.453 ms	0.380 ms	1.19x faster
CPU Duty Cycle (@ 100 Hz)	4.53%	3.80%	-16.1%
Energy per Inference	44.89 µJ	37.66 µJ	-16.1%

The 77.6% model memory reduction and 67.8% total SRAM reduction enable comfortable deployment on the ESP32, utilizing only 3.9% of available SRAM. The slight degradation in accuracy metrics is offset by the improvement in recall (+0.42 pp), the primary metric for gesture detection sensitivity.



9.2 Summary of Contributions

This report documents two enhancements to the Smart Ankleband gesture classification system, applied sequentially to improve both classification performance and deployment efficiency on the ESP32 microcontroller.

In the first phase, we evaluated 86 filter configurations across six filter families and found that EMA filtering with α=0.3 provides the best improvement for downstream CNN classification, increasing gesture recall from 88.33% to 89.83% (+1.50 pp).

In the second phase, we applied structured pruning at 40%, reducing the model from 56.36 KB to 38.32 KB with negligible performance impact. Physical neuron removal by reconstructing smaller weight tensors rather than relying on PyTorch's weight masking was necessary to achieve real memory savings on the device. Subsequent INT8 quantization-aware training reduced the model further to 12.60 KB, with only a 0.43 percentage point reduction in recall. Batch normalization layers were retained in FP32 to preserve classification accuracy.

Together, the three stages yield a model that is 77.6% smaller than the original baseline while maintaining recall above baseline levels, demonstrating that the system can be meaningfully improved within the hardware constraints of the ESP32.

References

[1] Zadok, S., Yona, G., Karasik, R., Shpunt, A., & Plotnik, M. (2024). Smart Ankleband for Plug-and-Play Hand-Prosthetic Control Using Deep Learning. *IEEE Transactions on Neural Systems and Rehabilitation Engineering*. Technion - Israel Institute of Technology.

[2] Li, H., Kadav, A., Durdanovic, I., Samet, H., & Graf, H. P. (2017). Pruning Filters for Efficient ConvNets. *International Conference on Learning Representations*.

[3] Molchanov, P., Mallya, A., Tyree, S., Frosio, I., & Kautz, J. (2019). Importance Estimation for Neural Network Pruning. *IEEE Conference on Computer Vision and Pattern Recognition*, 11264-11272.

[4] Zhu, M., & Gupta, S. (2018). To Prune, or Not to Prune: Exploring the Efficacy of Pruning for Model Compression. *International Conference on Learning Representations Workshop*.